

Performance Profiling and Bottleneck Isolation in a Java-Based Simulation Server

Technical Performance-Debugging Documentation

June 2026

Abstract

This document describes what has been done to optimize the tick rate and chunk-loading rate of a heavily-modded Minecraft: Java Edition modpack which was developed for use on the server. The failure being observed was not the graphics frame rate issue; it was the server-side simulation delay. The symptoms included slow region generation, overload warning alerts at server threads, and entity responses delay during testing.

The data collection was performed through runtime logs, Spark profiling data, configuration files, subsystem removal tests, and CSV summary files. For the high-load test I chose a 32-chunk render distance and 32-chunk simulation distance. During testing, I moved forward to load regions and attacked entities to see their delay. Seed numbers were used as a nuisance variable, though the trend observed was more correlated with the subsystems/ configurations used rather than with the seed change. Therefore, it would be impossible for the retrospective dataset to separate all the factors; the causal claims would be limited.

The strongest result is that the number of overload warnings has decreased from 1.52/min to 0.14/min according to the runtime logs. The maximum tick delay has gone down from 48,576 ms to 3,978 ms, and the total accumulated tick delay from 311,200 ms to 6,259 ms. Therefore, it is reasonable to conclude that the main bottleneck in this case was the accumulated server thread workload caused by the region generation, structure/template systems, entities/spawning systems, AI/pathfinding behavior, saving/loading processes, and configuration.

Contents

1	Introduction	2
2	Project Objective and Scope	2
3	Test Environment and Controls	2
4	Measurement Method	4
5	Runtime Log Results	4
6	Spark Profiler Results	5
7	Subsystem Evidence and Interpretation	6
8	Chunk Radius Estimate	7
9	Configuration-Level Reductions	7
10	Evidence Sources	8
11	Discussion	9
12	Limitations	9
13	Conclusion	9
A	Representative Runtime Log Evidence	10
B	Symbol Summary	10
	References	11

1. Introduction

I created a heavily modded Minecraft: Java Edition modpack aimed at the server usage. Once the modpack was ready, the server experienced significant lags. The client was able to produce graphics frame rates, yet the server-side simulation lagged behind: chunks loaded slowly, entities froze or were late in reacting, and the server thread kept reporting the overload warnings.

My initial tests included changing the settings related to chunks and removing large content mods (e.g. Lycanites Mobs and boss systems). It helped me identify the cause, but not completely. Later, it turned out that it was not one particular mod or one world seed which caused the problem, but rather the server thread workload resulting from various subsystems and configurations.

This document outlines the performance debugging process that has been conducted on the server. The purpose was to identify which subsystem categories were most likely to cause tick lag and chunk loading lag and record the changes made to alleviate the observed server-thread overload.

For the troubleshooting procedure, the following software stack was used: Minecraft: Java Edition - the runtime environment, Minecraft Forge - the mod-loading platform, Spark - the profiler, FPS Overlay - the in-client FPS/TPS/MSPT visualization tool, and Java 17 - the runtime platform [1-5]. The major components that have been tested or configured are Lycanites Mobs, Tectonic, Dynamic Trees, Enhanced AI, When Dungeons Arise, Repurposed Structures, Towns and Towers, Moog's Voyager Structures, BetterEnd, BetterNether, The Aether, Deeper and Darker, The Deep Void, Blue Skies, The Graveyard, Alex's Mobs, Sound Physics Remastered, AmbientSounds, GeckoLib, Chunky, and Bosses Unleashed [6-26].

2. Project Objective and Scope

This documentation records the work completed to identify and reduce server-side lag in the modded Minecraft server. The work focused on three practical objectives:

1. Isolate graphics performance from server-thread simulation performance.
2. Identify the subsystem categories causing the highest tick and chunk loading pressure: region generation, structures, entities, AI/pathfinding, and save/load operations.
3. Contrast the runtime behavior before and after the changes in mods and configuration while taking into account the seed difference and different log durations.

The troubleshooting process revealed that the problem was the accumulation of server-thread work rather than graphics rendering. The decrease in generation pressure, AI/pathfinding workload, dynamic blocks/world updates, structure pressure, and spawn/event/dungeon activities yielded the most noticeable improvement in terms of repeated server-thread overload.

3. Test Environment and Controls

Only the variables that materially affected the investigation are listed below.

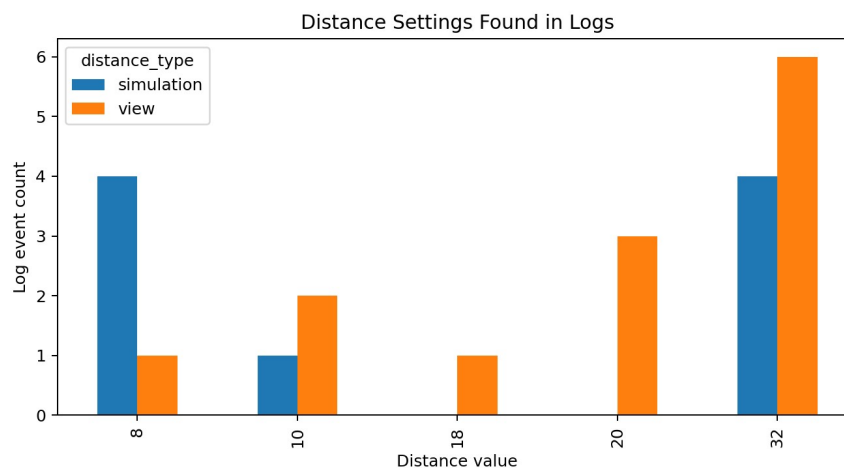
Table 1: Important test variables and controls.

Variable	Setting or status	Why it matters
CPU	AMD Ryzen 7 7800X3D	Tick simulation is CPU-sensitive.
GPU	AMD Radeon RX 7900 XT	Helped isolate rendering performance from server-thread performance.
RAM	32 GB total	Affects memory headroom and runtime stability.
Java/runtime	Java 17.0.15, Forge 47.4.20, Minecraft 1.20.1	Defines the runtime environment.
Players	1 in retained Spark captures	Isolates single-player server-thread performance.
Rendering distance	32 chunks during the high-load benchmark	Increases chunk pressure.
Simulation distance	32 chunks during the high-load benchmark	Increases simulation workload.
Benchmark action	Move forward to load regions; attack entities	Stresses chunk loading and entity responses.
Primary changed variables	Mod set and configuration values	Primary variables related to the observed improvement.
Seed	Varied	Nuisance variable.

The benchmark was designed to generate high loads. Render distance of 32 chunks and simulation distance of 32 chunks generated a vast active area. Movement generated additional loads on the server through new region loading, while entity attack tests highlighted issues like floating and delayed moving entities.

Table 2: Distance settings found in runtime logs.

Distance type	Values seen	High-load setting	Interpretation
View distance	8, 10, 18, 20, 32	32	32 chunks was the high-load condition.
Simulation distance	8, 10, 32	32	32 chunks was the high-load condition.

**Figure 1:** Distance-setting events extracted from runtime logs.

4. Measurement Method

A healthy Minecraft server targets 20 ticks per second. That gives a 50 ms budget per tick:

$$\tau_0 = \frac{1000 \text{ ms}}{20 \text{ ticks}} = 50 \text{ ms/tick.} \quad (1)$$

Here, 0 represents the target tick budget. When the processing of a tick takes more than 50 ms, the server falls behind. Formula (1) is directly applied to the Spark MSPT numbers and the server-thread overload chart.

Since the runtime logs collected in this study varied in terms of log time periods, it was necessary to normalize warning counts by time periods. Warning rate and delay rate metrics were defined as follows:

$$q_w = \frac{N_{\text{warnings}}}{T_{\text{minutes}}}, \quad (2)$$

$$q_d = \frac{D_{\text{total}}}{T_{\text{minutes}}}, \quad (3)$$

$$R = \frac{\text{Initial} - \text{Post-change}}{\text{Initial}} \times 100\%. \quad (4)$$

These are the formulas used in the results measurements presented in the documentation. No other formulas can be used except the simplified ones marked as estimations.

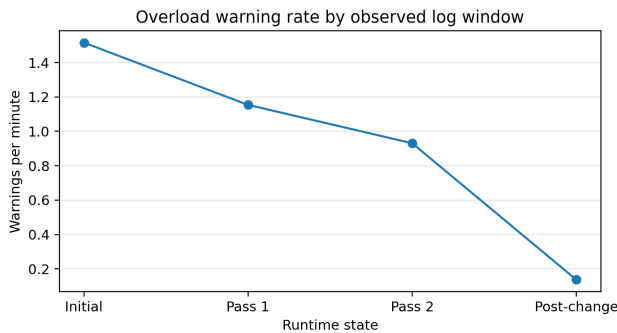
5. Runtime Log Results

This table shows the log time period used, which is calculated using the actual first and last timestamps of the runtime log file. This approach allows one to consider the duration of the different test log files.

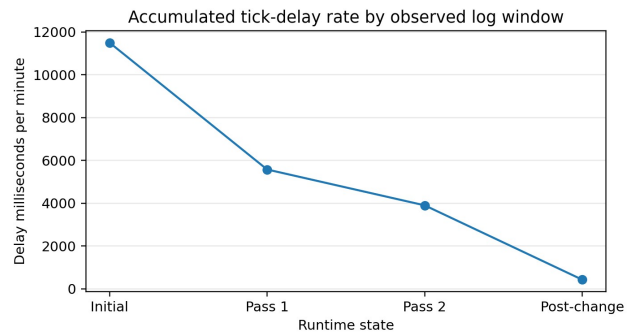
Table 3: Runtime metrics normalized by observed log window.

Runtime state	Window (min)	Warnings	Warnings/min	Max delay (ms)	Total delay (ms)	Delay ms/min
Initial heavy runtime state	27.05	41	1.52	48,576	311,200	11,504.6
After first removal/config pass	12.13	14	1.15	15,706	67,674	5,577.5
After additional removal/config pass	7.52	7	0.93	7,692	29,327	3,901.6
Post-change observed state	14.63	2	0.14	3,978	6,259	427.7

Normalized warning rate was reduced from 1.52 warnings per minute to 0.14 warnings per minute. Normalized total tick-delay rate was reduced from 11,504.6 ms per minute to 427.7 ms per minute. These metrics are more robust than just warning counts.



(a) Overload-warning rate.



(b) Accumulated tick-delay rate.

Figure 2: Normalized runtime-log metrics by observed log window.

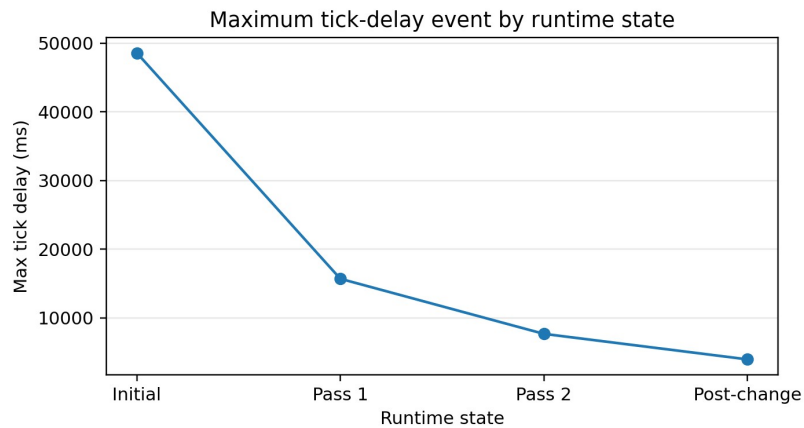


Figure 3: Maximum tick-delay event by runtime state.

6. Spark Profiler Results

The Spark logs also indicate extreme server-thread stall events. They provide great evidence about TPS/MSPT performance, but they alone do not prove improvement with each exact subsystem removal since the recorded profiler captures are not directly associated with an ideally controlled before/after matrix.

Table 4: Spark profiler measurements.

Capture	Duration	Ticks	TPS 1m	TPS 5m	Median MSPT	95th % MSPT	Max MSPT	Max/50 ms
General runtime capture	10m 20s	8,379	19.00	14.25	14.3	29.9	12,800	256.0x
Mod-view runtime capture	3m 43s	3,780	15.06	10.07	10.4	16.5	16,100	322.0x
Subsystem runtime capture	1m 1s	539	8.65	13.65	17.8	30.0	15,300	306.0x

The maximum MSPT values were several hundred times higher than the 50ms tick budget. This is consistent with the symptoms of delayed entity response and slower simulation update, despite continued frame rate rendering by the game.



Figure 4: Spark profiler record showing TPS/MSPT and profiler timeline evidence.

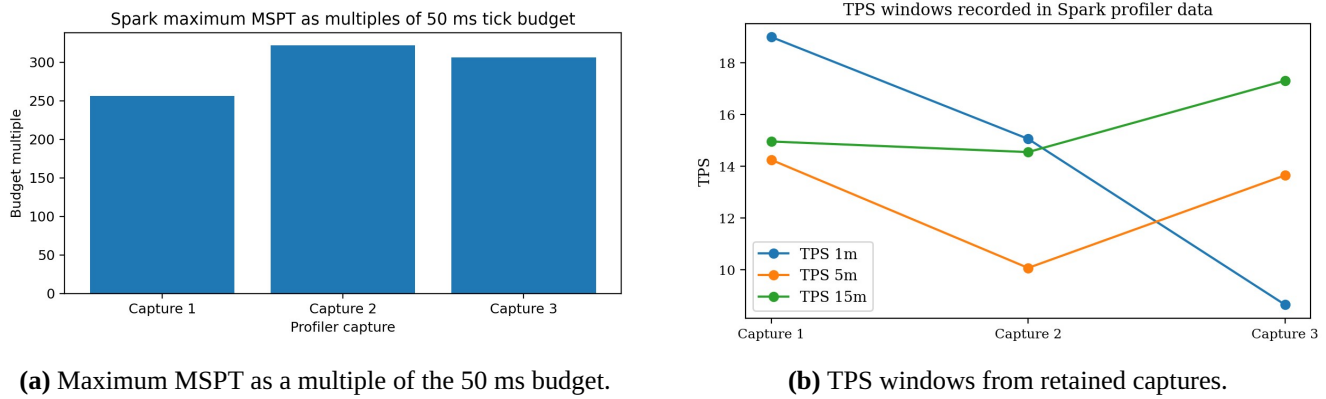


Figure 5: Spark profiler summaries.

7. Subsystem Evidence and Interpretation

Practical subsystem isolation was used for the investigation rather than the more scientific ablation experiment method. Ablation experiments would have required the repeat tests with identical seed, route, test duration, and a single subsystem change. See table 5 below for the best evidence for each subsystem without claiming causality.

Table 5: Subsystem evidence and interpretation.

Subsystem	Major examples	Main effect	Interpretation
Terrain and region generation	Tectonic	Changes terrain and region generation on entry into new chunks.	Major contributor to chunk-loading pressure.
Dynamic world/block systems	Dynamic Trees	Adds dynamic tree/block behavior and world update cost.	Contributing background tick workload.
AI/pathfinding	Enhanced AI	Changes the entity decision making and pathfinding behavior.	Contributing entity-response and tick workload.
Entity spawning, events, and dungeons	Lycanites Mobs configuration	Adds spawn points, events, dungeon spawners, and background event pressure.	Tuned instead of fully removed; likely workload contributor.
Structure/template generation	When Dungeons Arise; Repurposed Structures; Towns and Towers; Moog's Voyager Structures	Adds structure search, template pools, placement, and generation.	Contributing chunk-loading bottleneck.
Dimension/world-state systems	BetterEnd; BetterNether; The Aether; Deeper and Darker; The Deep Void; Blue Skies; The Graveyard	Adds dimensions, generation paths, world states, and save/load.	Increased total workload, but not the only isolated cause.
Additional entity/boss systems	Alex's Mobs; Bosses Unleashed	Adds entity and boss behavior.	Tested early; entity systems contribute, but not one large mob/boss mod as the only cause.
Client/audio/library warning noise	Sound Physics Remastered; AmbientSounds; GeckoLib	Produces client/audio/library warnings.	Mostly diagnostic noise, not the actual server-thread cause.
Pregeneration mitigation	Chunky	Pregenerate chunks ahead of time during runtime exploration.	Mitigation for runtime generation workload.

It was not enough to disable the largest mod. The data showed that the situation was much better explained by a com-

combination of high workloads due to terrain generation, structure processing, dynamic world updates, AI/pathfinding, entities/events, and saves/loads, all of which used the same tick budget.

8. Chunk Radius Estimate

The formula for the radius of chunks in meters is an approximation, not a CPU-time measure. The formula is presented here to show why 32 chunks can be considered a high-load scenario.

$$W(r) = (2r + 1)^2. \quad (5)$$

Table 6: Estimated active chunk positions by radius.

Radius	Estimated active positions
8	289
10	441
18	1,369
20	1,681
32	4,225

At 32 chunks, the estimate is 4,225 positions. That is about 14.6 times the 8-chunk estimate:

$$\frac{W(32)}{W(8)} = \frac{4225}{289} \approx 14.6. \quad (6)$$

This does not prove exact runtime cost, but it explains why moving forward at 32/32 was a strong stress test for chunk loading and region generation.

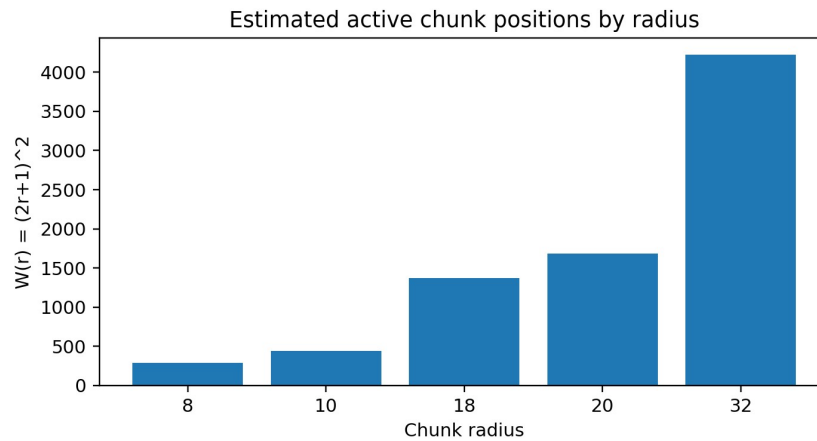


Figure 6: Estimated active chunk positions by radius.

9. Configuration-Level Reductions

The percentages in this section are configuration-level reductions. They are not direct measurements of CPU time, MSPT reduction, or total server workload reduction.

Configuration changes are interpreted using an after/before ratio:

$$r = \frac{x_1}{x_0}, \quad (7)$$

$$\Delta = (1 - r) \times 100\%. \quad (8)$$

For example, reducing spawnWeightScale from 0.065 to 0.02 gives

$$\frac{0.02}{0.065} \approx 0.3077, \quad \Delta \approx 69.2\%. \quad (9)$$

Table 7: Configuration-level reductions.

Setting	Baseline	Optimized	Interpretation
Tower / generation chance	0.25	0.01	96.0% lower chance parameter.
Per-type spawn limit	10	4	60.0% lower per-type cap.
Spawn weight scale	0.065	0.02	69.2% lower spawn-weight parameter.
Dungeon spawner weight scale	0.85	0.10	88.2% lower spawner-weight parameter.
Approximate dungeon density	distance = 44	distance = 192	Simplified estimate of about 94.7% lower spatial density.

The dungeon-density estimate assumes density scales approximately with inverse-square spacing:

$$R_\rho = \left(\frac{44}{192} \right)^2 \approx 0.0525, \quad \Delta_\rho \approx 94.7\%. \quad (10)$$

This is an approximating model, not an actual measurement of the inner workings of the mod's placement algorithm. The proper conclusion from this data is that these settings would reduce expected background activity, not that they could prove the percent decrease in CPU time spent on the server.

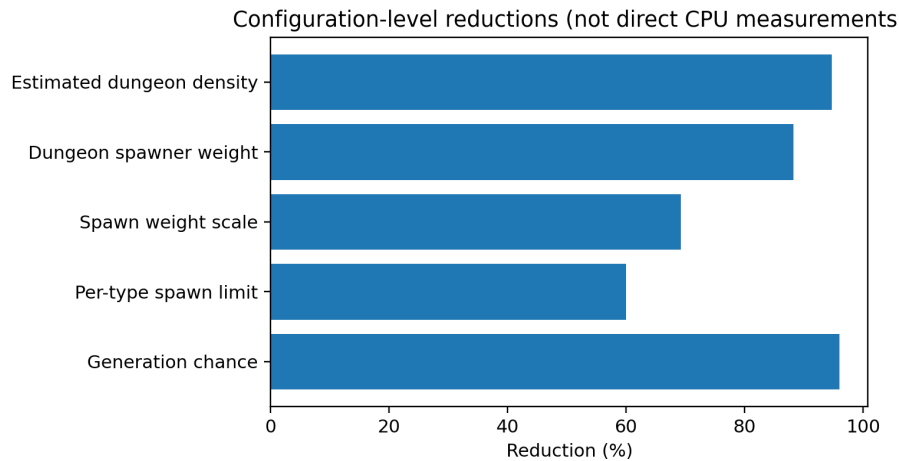


Figure 7: Configuration-level reductions; these are parameter changes, not direct CPU-time measurements.

10. Evidence Sources

There were four kinds of evidence: runtime logs, Spark profiler data, configuration files, and visual data. Runtime logs contained overload warnings, distance-setting statements, warnings related to the template pool, and save/load

data. Spark profiler data contained TPS, MSPT, time period, number of ticks, memory usage, and subsystem timing. Configuration files contained before/after settings. Visual data contained the difference between high frame rate and simulation rate.



Figure 8: Visual evidence showing high frame rate while simulation metrics were degraded.

11. Discussion

There is sufficient support for the modest systems theory approach. The server thread had too much built-up load, rather than a graphics issue. The most convincing supporting factor is the presence of slow chunk loading, delayed entity behavior, server-thread overload lines, Spark MSPT spikes, and decreased normalized warning/delay rates following subsystems and configuration changes.

Seed variation is an alternative hypothesis which cannot be entirely rejected since the selected dataset is a retrospective one. A different seed can influence local terrain generation and structure spawning. The observed trend is more in line with subsystems and configurations changes, but the retained dataset cannot completely exclude those possibilities.

The spark captures exhibit extremely high latency issues, but these are not the complete chain of causality. It proves the fact that there have been extreme MSPT spikes on the server thread. The runtime logs and configuration changes offer more convincing before and after comparison.

This documentation records the completed troubleshooting process, including Java server profiling, log parsing, configuration management, data visualization, and the methodological limits of the retained evidence.

12. Limitations

The current project is a retrospective analysis of completed troubleshooting and it does not make any claims towards journal level experimental controls. Multiple variables have changed during practical troubleshooting, and the retained data set lacks several p99 MSPT values, route coordinates and perfect before/after benchmarks for each subsystem.

The runtime logs comparison is based on observed time duration normalization but the duration comes from retained log timestamp windows rather than planned benchmark timing. Spark logs confirm the presence of severe lag, but they are not part of a completely controlled before and after data matrix. The configuration percentages are parameter reductions, not time reductions.

The intended use was server deployment, but the retained Spark captures used one player. Therefore, the retained data does not establish multiplayer scaling performance. It supports bottleneck isolation only under the retained test conditions.

13. Conclusion

The performance issue was a server-thread bottleneck. Graphics rendering was not the main problem. The strongest evidence came from normalized runtime log metrics, server-thread overload lines, Spark MSPT spikes, configuration changes, and subsystem testing.

Conclusion The strongest conclusion to draw is that there was an improvement in tick and chunk loading after alleviating the load from terrain generation, structure and template creation, world and block dynamics, AI/pathfinding load, and spawns/events/dungeons load. The above documentation does not prove causal relationships for each specific mod, but it does provide credible technical troubleshooting for a problem of server thread overload.

A. Representative Runtime Log Evidence

The retained logs include server-thread overload lines such as:

```
[22:15:14] [Server thread/WARN]: Can't keep up! Is the server overloaded?
Running 48576ms or 971 ticks behind
```

The retained logs also include distance-setting lines such as:

```
[22:05:03] [Server thread/INFO]: Changing view distance to 32, from 18
[22:37:59] [Server thread/INFO]: Changing simulation distance to 32, from 8
```

These lines support two central claims: the server thread was falling behind, and the high-load 32-chunk setting appeared in the retained runtime logs.

B. Symbol Summary

Table 8: Symbols used in the documentation.

Symbol	Meaning
τ	Measured milliseconds per tick.
τ_0	Target tick budget, 50 ms per tick.
$W(r)$	Approximate active chunk positions at radius r .
MSPT	Milliseconds per tick.
TPS	Ticks per second.
Warning rate	Overload warnings divided by runtime minutes.

References

- [1] Mojang Studios, “Minecraft: Java Edition.” [Online]. Available: <https://www.minecraft.net/en-us>
- [2] Minecraft Forge, “Forge downloads.” [Online]. Available: <https://files.minecraftforge.net/>
- [3] Lucko, “spark: A performance profiler for Minecraft clients, servers, and proxies.” [Online]. Available: <https://spark.lucko.me/>
- [4] Hichamdev, “FPS Overlay.” [Online]. Available: <https://modrinth.com/mod/fps-overlay>
- [5] Microsoft, “Microsoft Build of OpenJDK.” [Online]. Available: <https://www.microsoft.com/openjdk>
- [6] Lycanite, “Lycanites Mobs.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/lycanites-mobs>
- [7] Apollo, “Tectonic.” [Online]. Available: <https://modrinth.com/mod/tectonic>
- [8] Dynamic Trees Team, “Dynamic Trees.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/dynamic-trees>
- [9] Insane96, “Enhanced AI.” [Online]. Available: <https://modrinth.com/mod/enhanced-ai>
- [10] Aureljz, “When Dungeons Arise - Forge!” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/when-dungeons-arise>
- [11] TelepathicGrunt, “Repurposed Structures.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/repurposed-structures>
- [12] Kubek, “Towns and Towers.” [Online]. Available: <https://modrinth.com/mod/towns-and-towers>
- [13] finndog_123, “Moog’s Voyager Structures.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/moogs-voyager-structures>
- [14] Quiqueck, “BetterEnd.” [Online]. Available: <https://modrinth.com/mod/betterend>
- [15] Quiqueck, “BetterNether.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/betternether>
- [16] The Aether Team, “The Aether.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/aether>
- [17] KyaniteMods, “Deeper and Darker.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/deeperdarker>
- [18] The Deep Void Team, “The Deep Void.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/the-deep-void>
- [19] ModdingLegacy, “Blue Skies.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/blue-skies>
- [20] The Graveyard Team, “The Graveyard (Forge/NeoForge).” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/the-graveyard-forge>
- [21] AlexModGuy, “Alex’s Mobs.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/alexs-mobs>
- [22] henkelmax, “Sound Physics Remastered.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/sound-physics-remastered>
- [23] CreativeMD, “AmbientSounds.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/ambientsounds>
- [24] Gecko, “GeckoLib.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/geckolib>
- [25] pop4959, “Chunky (Forge/NeoForge).” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/chunky-pregenerator-forge>
- [26] hite1207, “Bosses Unleashed.” [Online]. Available: <https://www.curseforge.com/minecraft/mc-mods/bosses-unleashed>